e-puck2 Beginner Guide

First Steps for New Students

Prepared for Jack

July 21, 2026

Official starter link

The main official GCtronic e-puck2 guide is:

- https://www.gctronic.com/doc/index.php?title=e-puck2

Start there for manufacturer documentation, then use this beginner guide as the practical first-day roadmap.

What the e-puck2 is

The e-puck2 is a small differential-drive mobile robot used for robotics education and research. It is useful for teaching:

- basic programming,
- sensing and feedback,
- obstacle avoidance,
- control and estimation,
- multi-robot experiments.

For a beginner, the most important mental model is simple: the robot moves because its **left and right wheels** move at chosen speeds. If both wheels move equally, it goes roughly straight. If one wheel moves faster, it turns.

What beginners should learn first

Before writing advanced code, every student should be able to explain:

1. where the robot's forward direction is,
2. which wheels create turning,
3. what the proximity sensors are for,
4. why simulation should come before real-hardware testing,
5. how to stop a bad experiment quickly.

Safety rules for first-time users

Use these rules every time:

- Start in simulation whenever possible.
- Keep the robot on an open surface with space around it.
- Use low speeds for first tests.
- Test one change at a time.
- Be ready to lift the robot or switch it off if motion is wrong.
- Never run a new controller at full speed first.
- Do not assume sensor readings are perfect.

Recommended learning order

A beginner-friendly order is:

1. Understand the robot body, wheels, and basic sensors.
2. Run a simple e-puck2 simulation.
3. Try open-loop motion: forward, stop, turn.
4. Read sensor values.
5. Build a simple obstacle-avoidance behavior.
6. Only then move to the real robot.

Which simulation software to use

For beginners, use **Webots** as the main simulation software. It is the most natural first choice for e-puck2 learning because it already supports mobile-robot simulation well and is widely used for educational robotics workflows.

Recommended links:

- Webots getting started guide: Cyberbotics Webots guide
- GCtronic e-puck2 page: Official e-puck2 wiki

A simple beginner workflow is:

1. Install and open Webots.
2. Load an e-puck2 world or example project.
3. Run the simulation and watch how the robot moves.
4. Change a small controller constant.
5. Re-run and confirm the behavior changed.
6. Only after the simulation behaves well, try the same idea on the real robot at lower speed.

Basic starter code

Students should not start with a large project. Start with a tiny controller that moves forward, turns, and stops. Even pseudocode-level understanding is useful on day one.

Example beginner Python-style controller logic:

```
# basic idea: move forward for a while, then turn, then stop

left_speed = 2.0
right_speed = 2.0

# drive forward
set_wheels(left_speed, right_speed)
wait(2.0)

# turn in place
set_wheels(-1.5, 1.5)
wait(1.0)

# stop
set_wheels(0.0, 0.0)
```

What this teaches:

- equal wheel speeds produce roughly straight motion,
- opposite wheel directions produce an in-place turn,
- stopping is just setting both wheels to zero.

A first reactive behavior can be explained like this:

```
front_sensor = read_front_proximity()

if front_sensor > threshold:
    set_wheels(-1.0, 1.0)   # turn away
else:
    set_wheels(2.0, 2.0)    # move forward
```

This is enough to introduce the idea of **sense -> decide -> act**, which is the core robotics loop beginners need to understand.

Connection modes beginners should know

The e-puck2 can be used in several connection modes. For beginners, use **USB first**. It removes unnecessary wireless debugging problems.

USB: recommended first path

- Selector position: **8**

- Why use it first: simplest setup and easiest debugging
- Common Linux device names: `/dev/ttyACM0`, `/dev/ttyACM1`, `/dev/ttyACM2`

Typical first-use sequence:

1. Set the selector to **8**.
2. Connect one micro-USB cable.
3. Turn the robot on.
4. On Linux, make sure the user has serial permission if needed:

   ```
   sudo adduser $USER dialout
   ```

5. Log out and back in if the group was changed.
6. Check whether serial devices appeared:

   ```
   ls /dev/ttyACM*
   ```

Bluetooth

- Selector position: **3**
- Usually not the best first choice for beginners on Linux
- Can work well later, but pairing and serial binding add extra steps

WiFi

- Selector position: **15 (F)**
- Good when untethered control is needed
- Requires the correct WiFi firmware and network configuration

If the robot has no saved WiFi settings, it can create its own access point. A common first setup flow is:

1. Set selector to **15 (F)**.
2. Connect to the robot SSID `e-puck2_0XXXX`.
3. Use password `e-puck2robot`.
4. Open `http://192.168.1.1`.
5. Save the desired WiFi network credentials.
6. Power-cycle the robot once.

What to test on day one

A good first-day progression is:

1. Connect to the robot over USB.
2. Confirm the host computer can see the robot.
3. Read sensor values without moving the robot.
4. Blink an LED or perform another harmless check.
5. Command a tiny forward motion.
6. Command a tiny turn.
7. Stop and write down what happened.

Simple beginner experiments

These are appropriate first experiments:

- Drive straight for a short time.
- Turn left and right in place slowly.
- Stop when a front obstacle is detected.
- Compare left and right proximity readings near a wall.
- Run the same controller in simulation and then on the real robot at lower speed.

Common beginner mistakes

Watch for these:

- Going to real hardware too early.
- Testing at high speed first.
- Making many code changes before the next run.
- Forgetting to check selector position.
- Using a charge-only USB cable.
- Assuming USB port numbering will always stay the same.
- Confusing Bluetooth mode with WiFi mode.

What students should record in their lab notes

For each run, write down:

- date and robot ID,
- connection mode used,
- code version or script name,
- speed settings,
- expected behavior,
- actual behavior,
- next change to try.

A practical beginner checklist

Before saying "the robot is working," verify these basics:

- the selector is in the intended position,
- the robot powers on correctly,
- the host computer detects the device or network endpoint,
- a sensor read succeeds,
- a tiny motion command works,
- the robot can be stopped safely.

Bottom line

For beginners, the right workflow is: **learn the robot body → start in simulation → connect over USB → verify sensing → try tiny motions → only then attempt more advanced behaviors.**

That sequence prevents most frustrating first-day failures and gives students a clean path from curiosity to safe real-robot control.